



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Course Code: CSE 402.

Course Title: Operating System Lab.

Experiment No: 6

**Experiment Title: Implementation of Non-Preemptive and
Preemptive Priority CPU Scheduling Algorithm**

Date of Submission : 02-09-2026

Submitted By:

Name : Boishakhi Basu.

Student Id : 22201173

Semester : 4th Year 1st Semester

Section : C-2

Submitted To
Atia Rahman Orthi
Lecturer , UAP

Problem Statement:

Given the following processes with their **Arrival Time (AT)**, **Burst Time (BT)**, and **Priority**:

Process	AT	BT	Priority
P1	0	3	3
P2	1	4	2
P3	2	6	4
P4	3	4	6
P5	5	2	10

Implement the following CPU Scheduling Algorithms:

- **Priority Non-Preemptive Scheduling**
- **Priority Preemptive Scheduling** with a **Time Quantum (TQ)** of 1 unit

Assume that a **smaller priority number indicates a higher priority**.

For each algorithm, calculate **Completion Time (CT)**, **Turnaround Time (TAT)**, and **Waiting Time (WT)**. Also, calculate the **Average Turnaround Time (Average TAT)** and **Average Waiting Time (Average WT)**.

Finally, represent the **process execution sequence**, display the scheduling results for all processes, and **compare the performance of Preemptive and Non-Preemptive Priority Scheduling** to determine which algorithm performs better based on **Average TAT** and **Average WT**.

Source Code:

```
process = ["p1", "p2", "p3", "p4", "p5"]
at = [0, 1, 2, 3, 5]
bt = [3, 4, 6, 4, 2]
priority = [3, 2, 4, 6, 10]

n = len(process)
```

```

ct = [0] * n
wt = [0] * n
done = [0] * n
tat = [0] * n

time = 0
completed = 0

while completed < n:

    x = -1

    for i in range(n):

        if at[i] <= time and done[i] == 0:

            if x == -1 or priority[i] < priority[x]:
                x = i

            if x == -1:
                time += 1

            else:
                time = time + bt[x]

            ct[x] = time
            done[x] = 1
            completed += 1

    for i in range(n):
        tat[i] = ct[i] - at[i]
        wt[i] = tat[i] - bt[i]

print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):
    print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t", tat[i],
"\t", wt[i])

non_p_avg_tat = sum(tat)/n
non_p_avg_wt = sum(wt)/n

print("\nAverage TAT =", non_p_avg_tat)
print("Average WT =", non_p_avg_wt)

print("\n")

```

```

# Priority Preemptive

process = ["p1", "p2", "p3", "p4", "p5"]
at = [0, 1, 2, 3, 5]
bt = [3, 4, 6, 4, 2]
priority = [3,2,4,6,10]

n = len(process)

ct = [0] * n
wt = [0] * n
done = [0] * n
tat = [0] * n

tq = 1
remaining = bt.copy()
time = 0
completed = 0

while completed < n:

    x = -1

    for i in range(n):

        if at[i] <= time and remaining[i] > 0:

            if x == -1 or priority[i] < priority[x]:
                x = i

    if x == -1:
        time += 1

    else:
        if remaining[x] > tq:
            time += tq
            remaining[x] -= tq

        else:
            time += remaining[x]
            remaining[x] = 0
            ct[x] = time
            completed += 1

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

print("Process\tAT\tBT\tCT\tTAT\tWT")

for i in range(n):

```

```

        print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t", tat[i],
"\t", wt[i])

p_avg_tat = sum(tat)/n
p_avg_wt = sum(wt)/n

print("\nAverage TAT =", p_avg_tat)
print("Average WT =", p_avg_wt)

print("\n")

if non_p_avg_tat < p_avg_tat :
    print ("Non Preemptive Priority TAT is best.")
else:
    print ("Preemptive Priority TAT is best.")

if non_p_avg_wt < p_avg_wt :
    print ("Non Preemptive Priority WT is best.")
else:
    print ("Preemptive Priority WT is best.")

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Non Preemptive Priority WT is best.
PS D:\New folder> & C:\Users\lab\AppData\Local\Programs\Python\Python314\python.exe "d:/New folder/new.py
Process AT    BT    CT    TAT    WT
p1      0      3      3      3      0
p2      1      4      7      6      2
p3      2      6     13     11      5
p4      3      4     17     14     10
p5      5      2     19     14     12

Average TAT = 9.6
Average WT = 5.8

Process AT    BT    CT    TAT    WT
p1      0      3      7      7      4
p2      1      4      5      4      0
p3      2      6     13     11      5
p4      3      4     17     14     10
p5      5      2     19     14     12

Average TAT = 10.0
Average WT = 6.2

Non Preemptive Priority TAT is best.
Non Preemptive Priority WT is best.
PS D:\New folder>

```

Discussion:

This experiment examined how **Preemptive Priority Scheduling** and **Non-Preemptive Priority Scheduling** behave when applied to the same group of processes. The comparison focused mainly on the average turnaround time and average waiting time, as these values indicate how efficiently the processes are handled by the CPU.

The results show that the **Non-Preemptive Priority algorithm** achieved an Average TAT of **9.6** and an Average WT of **5.8**. In contrast, the **Preemptive Priority algorithm** produced an Average TAT of **10.0** and an Average WT of **6.2**. Since both performance values are lower in the Non-Preemptive approach, it required less overall waiting and completion time for this specific workload.

Although the Preemptive approach allows the CPU to switch when a higher-priority process arrives, such interruptions did not improve the overall results in this experiment. Therefore, the output demonstrates that the effectiveness of a scheduling method can depend on the arrival times, burst times, and priority values of the processes.

Conclusion:

Based on the experimental output, **Non-Preemptive Priority Scheduling performed better for the given process set**. It generated lower Average Turnaround Time (**9.6**) and Average Waiting Time (**5.8**) compared with the Preemptive Priority Scheduling results of **10.0** and **6.2**, respectively.

Thus, according to the selected performance measurements, the **Non-Preemptive Priority algorithm is the better choice for this particular experiment**. However, this does not mean that it will always outperform the Preemptive algorithm. The performance may change when different processes, priorities, arrival times, or burst times are used.

GitHub:

<https://github.com/boishakhi73/CSE-402-Operating-System-Lab>